

LLMxCPG: Context-Aware Vulnerability Detection Through Code Property Graph-Guided Large Language Models

Table of Contents

Introduction & Background

Methodology & Evaluation

Discussion & Conclusion

Introduction & Background

01

Introduction & Background

기존 딥러닝 기반 취약점 탐지의 한계

- 함수 단위 분석: 함수 호출 관계나 넓은 컨텍스트를 놓침
- 검증된 데이터셋에서 성능이 급락
 - PrimeVul에서 검증했을 때 SOTA 모델들의 정확도가 최대 45%까지 떨어짐 → 표면적 특징을 학습했을 가능성
 - 의미 보존 변형만으로도 모델 성능이 흔들림
- 컨텍스트 윈도우의 제약: 긴 코드를 한꺼번에 다루지 못함

CPG (Code Property Graph) 개념

- 여러 코드 표현을 하나의 그래프 구조로 병합
 - AST (구문 구조), CFG (제어 흐름 정보), PDG (제어 및 데이터 의존성)을 결합해, 복잡한 패턴을 graph traversal로 표현

LLMxCPG의 제시

CPG와 LLM을 결합한 방법론으로 두 단계로 구성됨

- CPG 기반 슬라이스 구축: 입력 코드를 정밀한 취약점 중심 코드 슬라이스로 추출
- 슬라이스 기반 LLM 파인튜닝: 노이즈가 제거된 간결한 슬라이스로 LLM을 파인튜닝하여, 표면적 특징이 아닌 본질적 취약점 패턴을 학습

Methodology & Evaluation

02

Methodology

LLMxCPG 전체 개요

- LLMxCPG는 입력 코드를 받아 취약점 여부(Vulnerable / Safe)를 출력하는 2단계 구조
- 두 개의 모델로 구성
 - LLMxCPG-Q (Qwen2.5-Coder-32B-Instruct 기반): 잠재적으로 취약한 실행 경로(execution path)를 식별하는 CPGQL 쿼리를 생성
 - LLMxCPG-D (QwQ-32B-Preview 기반): 추출된 slice를 보고 취약 여부를 분류
- 핵심: 코드를 통째로 분석하지 않고, 취약점과 관련된 핵심 부분만 잘라낸 뒤 분류

Methodology

Program Slicing — 개념

- 데이터 흐름 및 제어 흐름 분석을 통해 프로그램을 더 작은 표현으로 축소하는 기법
- 결과물(slice)은 원본 프로그램을 특정 동작 영역 내에서 충실히 대표하는 독립적 프로그램
- Criterion point: 불안정한 변수(insecure variable)나 불안정한 함수 호출(insecure function call)을 포함하는 코드 줄 (예: 검증 없이 흘러가는 입력값, strcpy 등)
- Criterion point가 식별되면, 두 방향으로 slice를 구성
 - Backward slicing: criterion point의 동작에 영향을 미칠 수 있는 모든 코드 요소 포착
 - Forward slicing: criterion point가 영향을 미칠 수 있는 모든 코드 요소 식별
- 기존 Program Slicing의 문제: 적절한 criterion point 선택의 어려움, 결과 slice에 불필요한 코드가 포함

Slice Construction — 이 연구에서 사용하는 slicing의 3단계 절차

실행 경로(execution path)에 집중. CPG를 활용하여 잠재적으로 취약한 실행 경로를 식별해 본질적 흐름을 파악.

- 1단계 — Taint Path
 - 목표 취약점과 관련된 실행 경로를 추출하는 CPGQL 쿼리를 설계
 - 쿼리 생성을 위해 fine-tuning한 모델(=LLMxCPG-Q)을 이용
- 2단계 — Interacter
 - 추출한 경로 자체만으로는 불완전하기 때문에 컨텍스트 정보를 찾음
 - LLMxCPG-Q가 생성한 두 번째 쿼리는 실행 경로와 상호작용하는 모든 식별자를 찾음 (라인 번호가 일치할 때 interacter로 간주)
- 3단계 — Backward Slicing
 - 실행 경로 + interacter 전부에 backward slicing을 적용해 완전한 코드 조각을 생성
 - 잠재적 취약점을 이해하는 데 필요한 모든 컨텍스트를 담은 간결한 슬라이스

Methodology

모델 구현 — LLMxCPG-Q (쿼리 생성)

- Qwen2.5-Coder-32B-Instruct를 CPGQL 쿼리 생성용으로 fine-tune (= LLMxCPG-Q)
- 학습 데이터는 DeepSeek-v3로 초기 쿼리를 생성하고 Joern 서버에서 검증한 뒤 이용
 - 실패한 쿼리는 에러 메시지를 피드백으로 주어 반복적으로 다듬는 방식으로 구축
 - 두 번의 추가 시도를 허용하고 세 번째에도 유효하지 않으면 폐기
- 여러 repository의 병렬 처리를 가능하게 하기 위해 Docker 컨테이너로 Joern 서버 클러스터를 배포

모델 구현 — LLMxCPG-D (분류)

- QwQ-32B-Preview를 분류용으로 fine-tune한 모델이 LLMxCPG-D
- Fine-tuning 데이터는 학습 데이터셋의 ground-truth 레이블을 기반으로 LLMxCPG-Q를 돌려 추출한 vulnerable / safe 슬라이스로 구성
- 추론 파이프라인: LLMxCPG-D는 yes/no만 판별하면 되므로 head를 축소
 - 축소된 head는 "Yes"(=Vulnerable)와 "No"(=Safe) 토큰에 해당하는 가중치 벡터로 구성

Fine-Tuning 방법

- Fine-tuning 방법
 - LoRA (Low-Rank Adaptation) 사용, rank 8 / alpha 4, learning rate 10^{-4}
 - LLaMA-Factory 프레임워크 사용
 - NVIDIA A100-80GB GPU (Ubuntu 22.04)에서 학습

Evaluation

데이터셋

- Training Dataset
 - FormAI-v2: LLM(GEMINI-pro, GPT-4, Falcon 180B 등)으로 생성한 331,000개의 C 프로그램. ESBMC 기반 formal verification으로 라벨링
 - PrimeVul: 라벨 정확도를 human-verified benchmark까지 끌어올린 데이터셋. 228,800개 safe + 6,968개 vulnerable, 140개 CWE 커버, 80/10/10 chronological split
- 평가 데이터셋 (Generalizability용 — 학습에 미사용)
 - SVEN: GitHub 보안 패치에서 수작업으로 큐레이션한 1,600여 개 C/C++/Python 프로그램
 - ReposVul: 1,491개 프로젝트, 236개 유형 CWE, 6,134개 CVE 항목 커버
- 대상 CWE 범위
 - 정적 CPG 분석으로 다룰 수 있는 메모리 계열: CWE-119, 120, 121, 122, 125, 190, 415, 416, 787
 - Race condition 같은 동적 행동 의존 취약점은 의도적으로 제외
- 데이터셋별 CWE 분포
 - FormAI-v2: 4개 CWE 유형 (119, 190, 415, 416) vulnerable 5,893개 (CWE당 1,395~1,500개, 비교적 균형), safe 4,431개 (ESBMC 구성 하에서 검증)
 - PrimeVul: 학습 데이터에 8개 CWE 유형 포함 SVEN (테스트): 4개 CWE 유형 (125, 190, 416, 787) CWE당 paired vulnerable/safe 37~122쌍
 - ReposVul: 7개 CWE 유형에 걸친 matched safe/vulnerable 균형 부분집합

Evaluation

쿼리 생성 성능

1,278개 테스트 샘플에 대한 유효 CPGQL 쿼리 생성 개수

모델	유효 쿼리 수
Qwen2.5-Coder-32B-Instruct (base)	19
DeepSeek-v3	132
LLMxCPG-Q (fine-tuned)	1,278

보안 전문가 3명이 50개 쿼리를 수작업 검증, true positive/negative 샘플의 76%에서 쿼리가 취약점 패턴과 의미적으로 매칭됨 (Fleiss' Kappa 0.6429)

Evaluation

코드 감소 효과

프로젝트 단위에서도 의미 있는 결과를 보임

데이터셋	코드 감소율
FormAI (합성)	78.70%
PrimeVul (함수 단위)	67.84%
SVEN (함수 단위)	70.22%
ReposVul (프로젝트 단위)	90.93%

Evaluation

함수레벨 분석

- function-level 코드 스니펫에서 취약점 탐지에 대해 높은 성능
- CWE별 성능 — 충분한 학습 샘플이 있는 메모리 계열에서 강세
 - CWE-119 (Buffer Overflow): F1 0.970
 - CWE-416 (Use-After-Free): F1 0.848
 - CWE-415 (Double Free): F1 0.852
 - CWE-190 (Integer Overflow): F1 0.792
- 데이터셋별 성능
 - FormAI — Accuracy 0.8146 / Precision 0.8097 / Recall 0.8054 / F1 0.8075
 - PrimeVul — Accuracy 0.7250 / Precision 1.000 / Recall 0.450 / F1 0.6206

Evaluation

SOTA 비교 — SVEN (미학습 데이터셋)

Model	Accuracy	F1
VulSim	0.33	0.31
VulBERTa-CNN	0.50	0.44
VulBERTa-MLP	0.50	0.43
ReGVD	0.51	0.55
LLMxCPG	0.6020	0.7048

SOTA 대비 Accuracy 약 +20%, F1 +15%p 이상 향상. 학습하지 않은 데이터셋에서도 LLMxCPG는 의미 있는 탐지를 해낸다.

Evaluation

프로젝트 수준 탐지

- 프로젝트 수준의 실세계 데이터로 학습되지 않았음에도 불구하고 평가를 수행
 - ReposVul — Accuracy 0.634 / F1 0.610
 - PKCO-25 (2025 신규 CVE) — Accuracy 0.600 / F1 0.617
- ReposVul 샘플 중 LOC가 1,623줄에 달하는 극단적으로 긴 샘플에서도 0.83의 정확도를 유지 → 코드 크기에 견고함
- 학습 cutoff 이후 CVE에서도 안정적 성능을 보여 본질적 취약점 특성을 학습했음을 시사한다

Evaluation

오분류 분석

- 잘 잡는 CWE: CWE-119, 190, 415, 416
- 못 잡는 CWE (정적 분석의 구조적 한계)
 - CWE-120 (Classic Buffer Overflow): F1 0.20
 - CWE-125 (Out-of-bounds Read): F1 0.46
- 원인 분석
 - 학습 데이터에 해당 CWE 샘플이 적음 (PrimeVul 기준 CWE-120 35개, CWE-125 391개) — 심한 클래스 불균형
 - 미묘한 메모리 접근 패턴(예: off-by-one)은 포착하기 어려움
- ReposVul과 SVEN 데이터셋이 제기하는 도전에도 불구하고, 우리 모델은 유망한 일반화 능력 — 0.6을 초과하는 정확도 점수

Evaluation

코드 변환 견고성

코드가 의미 보존 변환을 거칠 때 어느 정도 성능을 잘 유지함. 4가지 의미 보존 변환을 적용.

- T1 (Identifier Renaming): 모든 함수 파라미터를 random token으로 rename
- T2 (Statement Insertion): 실행되지 않는 코드 삽입
- T3 (Statement Reordering): 함수 본문을 별도 함수로 분리
- T4 (Statement Removal): 모든 주석 제거
- 결과
 - T4 (주석 제거)에 완전 무영향 — slice 구축 자체가 실행 경로 중심이라 주석을 애초에 무시
 - 데이터셋별 robustness 차이 — FormAI에서 가장 견고하며, 변환 후 F1이 오히려 약간 상승(+2.10%)하기도 함
 - T3가 가장 까다로움 — 슬라이스 기반 접근법이 함수 경계의 변화에 다소 민감
 - PrimeVul에서는 변환 시 precision은 감소하지만 recall이 상승 — 모델이 더 보수적으로 vulnerable을 예측

Discussion & Conclusion

03

Discussion & Conclusion

한계점

- CPG로는 프로그램 동작이나 복잡한 아키텍처 결정을 포착할 수 없음
- 고품질 데이터셋이 부족해 학습이 어려움
- 이진 분류의 설명 부족
 - reasoning 과정이 불투명함
 - DeepSeek-v3의 reasoning trace로 fine-tuning을 시도했으나 개선이 없었음
- Context length의 실용 제약
 - LLMxCPG-Q는 자원 한계로 32K 토큰으로 fine-tune되어, 슬라이싱 전 단계에서 매우 큰 commit이나 파일은 한 번에 처리하기 어려움
 - LLMxCPG-D는 슬라이스만 받으므로 8K로도 충분

결론

- F1-score는 최대 40% 향상
- 함수 단위 학습으로 프로젝트 단위 일반화
- 코드 변형에 대한 robustness 실증